Pruebas de ataque: hombre en el medio e inyección SQL

SSGUI · Sistema de Solicitud de Gestión de Usuarios Interno · ANDINA S.A.S. (empresa ficticia; todos los datos son ficticios)
José Bedoya, Carlos Ruiz, Luis Cerón y Cristian Caro · Institución Universitaria Politécnico Grancolombiano · Taller de Desarrollo Seguro de Software · Grupo G01
Docente: José Eduardo Patiño Santafé · Entregable 5 de 5 · 9 de noviembre de 2026

1. Resumen

Este documento presenta los resultados de dos ataques dirigidos contra el sistema SSGUI, pedidos por la actividad del curso para demostrar los controles de **confidencialidad, autenticidad e integridad** de la información: un ataque de **hombre en el medio** (interceptación del tráfico, *MitM*) y una **inyección SQL** (*SQL Injection*). Las pruebas se realizaron sobre la arquitectura de referencia, que es local (BunkerWeb como cortafuegos de aplicaciones web, la aplicación Django, PostgreSQL 17, GLPI 11 y sus redes internas de Docker).

El resultado global es favorable: ningún parámetro resultó inyectable, el cortafuegos bloqueó las cargas de inyección evidentes, un interceptor que presenta un certificado propio es rechazado por un cliente que valida la cadena, no se puede forzar una versión antigua de TLS y la contraseña de un inicio de sesión no viaja en claro por el tramo externo. Las dos debilidades conocidas son de tipo interno —el tramo entre el trabajador y GLPI y el tramo entre GLPI y su base de datos no usan TLS— y ya están registradas (B-30, B-32); se apoyan en el aislamiento de red y en que ningún contenedor distinto del cortafuegos publica puertos.

Tabla 1. Resumen de pruebas y resultados

Ataque	Posición	Herramienta	Resultado
Inyección SQL	Ocho objetivos, directo a la aplicación, nivel 5, con la cabecera Host y segundo orden	sqlmap 1.10.10	0 parámetros inyectables (siete objetivos con veredicto; el de segundo orden, ver §4)
Inyección SQL	Cinco cargas clásicas a través del cortafuegos	curl + OWASP CRS	Las cinco bloqueadas con 403
Inyección SQL	Cargas clásicas directo a la aplicación (sin cortafuegos)	curl + ORM de Django	Tratadas como dato literal (200); una sobredimensionada la rechaza la validación (404); 0 errores de base de datos; las 26 tablas intactas
Hombre en el medio	Interceptor con certificado falso frente a un cliente que valida	openssl + curl	El cliente que valida lo rechaza (error de certificado); solo un cliente comprometido lo acepta
Hombre en el medio	Degradación de TLS (anti-downgrade)	curl `--tls-max 1.1`	Rechazada: el borde solo ofrece TLS 1.2 y 1.3
Hombre en el medio	Despojo de TLS (anti-SSL-strip)	curl	El puerto 80 redirige a 443 y el borde envía HSTS de un año
Hombre en el medio	Robo de la cookie de sesión	curl	La cookie lleva Secure, HttpOnly y SameSite
Hombre en el medio	Captura del tramo navegador<->cortafuegos	tcpdump + tshark	Una contraseña centinela no aparece en claro (va dentro de TLS)
Capa TLS del borde	testssl.sh	testssl.sh 3.2.4	Solo TLS 1.2 y 1.3; ninguna de las vulnerabilidades probadas

2. Alcance, reglas y precauciones

Todas las pruebas se ejecutaron sobre el estado actual del *stack* local, a través del cortafuegos (la defensa) y, en los casos indicados, directamente contra la aplicación dentro de su red interna (el peor caso, con el cortafuegos retirado), para separar las dos capas de defensa.

Se aplicaron las mismas precauciones que en el entregable 2:

1. **Neutralización previa.** Antes de sqlmap se creó una copia de la base de datos y se detuvo el trabajador de integración, para que las solicitudes enviadas por el escáner no generaran tickets en GLPI. Las pruebas de hombre en el medio y las manuales son de solo lectura y no modifican datos.
2. **Restauración posterior.** Al terminar se restauró la base y se comprobó que el número de filas coincidiera con el inicial (818 registros de auditoría, 33 solicitudes, 148 tickets y 87 colaboradores en este *stack*).

3. **Verificación de integridad.** Antes de restaurar se comprobó que la cadena de auditoría encadenada por *hash* siguiera siendo válida (1025 registros, cadena válida, sin registro afectado).
4. **Control positivo.** Las herramientas de inyección se validan contra una aplicación de laboratorio deliberadamente vulnerable (entregable 2, §8): sqlmap sí encuentra la inyección sembrada. Esto descarta que un «0 inyectables» se deba a una herramienta mal configurada.

3. Las tres propiedades y los controles que las sostienen

La actividad pide controles de confidencialidad, autenticidad e integridad. El sistema los sostiene así, y cada prueba de este documento ataca uno o varios:

Propiedad	Amenaza principal	Control	Prueba que lo ejercita
Confidencialidad	Un atacante en la red lee las credenciales o los datos	TLS 1.2/1.3 en el borde; cookie con Secure y HttpOnly; aislamiento de las redes internas	§5.3, §5.5
Autenticidad	Un atacante se hace pasar por el servidor (interceptación) o inyecta una identidad	Certificado validado por el cliente; HSTS; sesión solo tras contraseña y TOTP (AAL2)	§5.1, §5.2, §5.4
Integridad	Un atacante altera los datos o la consulta (inyección)	ORM parametrizado; validación del formulario en el servidor; cortafuegos con OWASP CRS; auditoría encadenada por *hash*	§4

4. Inyección SQL

4.1 sqlmap, nivel 5, directo a la aplicación

Se ejecutó sqlmap en nivel 5 (el más agresivo) contra ocho objetivos, incluida la prueba de la cabecera `Host` y una prueba de segundo orden (una carga que se guarda en una solicitud y se busca después al listarla). Los objetivos cubren las entradas de cada rol: búsqueda de cuentas y ficha del técnico, listado y detalle de RRHH, auditoría del auditor, administración de usuarios del administrador, el formulario de inicio de sesión sin sesión y el registro de un retiro.

Tabla 2. Resultado de sqlmap por objetivo (nivel 5)

Objetivo	Rol	Parámetros inyectables
`GET /cuentas`	Técnico	0
`GET /colaboradores/<id>`	Técnico	0
`GET /solicitudes?pagina`	RRHH	0
`GET /solicitudes/<id>`	RRHH	0
`GET /auditoria?pagina`	Auditor	0
`GET /administracion/usuarios`	Administrador	0
`POST /ingreso`	Sin sesión	0
`POST /solicitudes/nueva/RET` (segundo orden)	RRHH	sin veredicto en esta corrida (ver nota)

Siete de los ocho objetivos llegaron a la conclusión de sqlmap «todos los parámetros probados no parecen inyectables» (0 inyectables), incluidas las cabeceras `Host`, `User-Agent` y `Referer`. El octavo, el de segundo orden, no alcanzó un veredicto final porque la corrida superó el tope de treinta minutos del protocolo de registro: con la opción `--second-url`, sqlmap compara dos páginas por cada petición y el formulario de Django, que valida y rechaza los datos malformados antes de cualquier consulta, hace que esa comparación sea muy lenta; la lentitud es de la herramienta, no una señal de vulnerabilidad. El mismo objetivo de segundo orden ya se había resuelto con 0 inyectables en la corrida documentada en el entregable 2 (`F6-04-SQLMAP-NIVEL5`), y en las cargas manuales de la sección siguiente el segundo orden se descarta de otra forma.

Durante la corrida sqlmap marcó de forma transitoria algún parámetro de cabecera como «posiblemente inyectable» con su prueba heurística inicial, pero lo retiró en su veredicto final tras las comprobaciones; es un falso positivo habitual por páginas de contenido dinámico, ya discutido en el entregable 2. La cadena de auditoría quedó íntegra (1025 registros, válida) y la base se restauró sin cambios.

4.2 Cargas manuales

Para complementar al escáner se enviaron cargas clásicas de inyección en el parámetro `documento` de la búsqueda de cuentas, en dos posiciones.

A través del cortafuegos. Las cinco cargas (`1' OR '1'='1`, `1; DROP TABLE app.solicitud--`, `' UNION SELECT null,version()--`, `1) OR (1=1`, `admin'--`) fueron **bloqueadas con 403** por el conjunto de reglas OWASP CRS antes de llegar a la aplicación.

Directo a la aplicación (sin cortafuegos), con una sesión de técnico. Las cargas que caben en el campo (máximo 20 caracteres) llegan a la consulta del ORM de Django, que las trata como un **dato literal**: la respuesta es 200 sin coincidencias y **sin ningún error de base de datos** ni fuga de información. Una carga más larga que el campo la rechaza la validación del formulario con 404, antes de tocar la base. Al terminar, las **26 tablas** del esquema seguían presentes: ningún `DROP TABLE` tuvo efecto.

La defensa de fondo es doble: el ORM de Django parametriza toda consulta (nunca construye SQL con cadenas), y los formularios validan cada entrada en el servidor. El cortafuegos añade una capa que detiene las cargas evidentes antes de la aplicación.

5. Hombre en el medio (interceptación)

Un ataque de hombre en el medio busca colocarse entre el navegador y el servidor para leer o alterar el tráfico. Se probaron las cinco vías habituales.

5.1 Suplantación del servidor con un certificado falso

El atacante levantó un servidor TLS con su propio certificado autofirmado para el nombre `app.localhost` (como haría un interceptor). Un cliente que **valida la cadena** contra el certificado real del borde **rechaza la conexión** con un error de certificado no confiable (código 60 de curl). Solo un cliente **comprometido**, que ignora la validación (`curl -k`), acepta al interceptor y recibe 200. Es la base de la defensa contra el hombre en el medio: la autenticidad del servidor se comprueba con el certificado, de modo que interceptar exige además comprometer la confianza del cliente, no basta con estar en la red.

> En la demostración local el certificado es autofirmado; en un despliegue con una autoridad pública (por ejemplo Let's Encrypt) la validación sería automática en cualquier navegador. La conclusión no cambia: un cliente que valida detecta la suplantación.

5.2 Degradación de la versión de TLS

Un intento de forzar `TLS 1.1` o anterior (`curl --tls-max 1.1`) **falla el saludo**: el borde solo ofrece TLS 1.2 y 1.3. Esto cierra los ataques que dependen de versiones antiguas. El inventario completo con `testssl.sh` (entregable 2 y evidencia `F7-07-TESTSSL`) lo confirma: solo TLS 1.2 y 1.3; sin SSLv2/SSLv3/TLS 1.0/1.1; **no vulnerable** a Heartbleed, ROBOT, POODLE, BEAST, SWEET32, LOGJAM ni RC4; firma ECDSA con SHA-256.

5.3 Despojo de TLS (SSL-strip)

Un ataque de despojo intenta mantener a la víctima en HTTP para leer el tráfico. El puerto 80 **redirige a 443** (301) y el borde envía la cabecera **HSTS** `max-age=31536000; includeSubDomains`: tras la primera visita, el navegador se niega a usar HTTP con ese sitio durante un año.

5.4 Robo de la cookie de sesión

Si la cookie de sesión viajara por HTTP o fuera legible por un *script*, un atacante en la red o por XSS podría robar la sesión. La cookie lleva los tres atributos de protección: **Secure** (nunca se envía por HTTP), **HttpOnly** (no la lee JavaScript) y **SameSite=Lax** (no se envía en peticiones de otros sitios). La sesión, además, solo se crea tras verificar la contraseña y el segundo factor TOTP (AAL2).

5.5 Captura del tramo navegador<->cortafuegos

Se capturó el tráfico del tramo externo con tcpdump mientras se enviaba un inicio de sesión con una **contraseña centinela**. Al analizar la captura con tshark, la contraseña **no aparece ni una vez en claro**: viaja dentro del túnel TLS. El dato sensible queda protegido en el único tramo que un atacante externo podría observar.

5.6 Tramos internos (debilidad conocida)

Los tramos **internos** entre contenedores no usan TLS: el trabajador habla con GLPI por HTTP (con el *token* de la API visible dentro de la red interna, B-32) y GLPI habla con su base de datos sin cifrar (B-30). No son alcanzables desde fuera del equipo: solo el cortafuegos publica puertos (verificado con nmap en el entregable 2), y cada base está en una red interna a la que solo llega su consumidor. El riesgo de un hombre en el medio interno exige que el atacante ya esté dentro de la red de Docker del anfitrión. Está registrado y aceptado como deuda, con la opción de TLS mutuo interno como trabajo futuro.

6. Relación con los controles propuestos

Control del entregable 4	Prueba de este documento	Veredicto
Cifrado en tránsito en el borde (TLS)	§5.2, §5.5, testssl	Cumple
Autenticación del servidor (certificado)	§5.1	Cumple
Anti-despojo (redirección + HSTS)	§5.3	Cumple
Protección de la sesión (atributos de la cookie, AAL2)	§5.4	Cumple
Prevención de inyección (ORM, validación, cortafuegos)	§4	Cumple
Integridad de la auditoría tras el ataque	§2, §4.1	Cumple
Cifrado en tránsito interno	§5.6	No cumple (B-30, B-32; aceptado)

7. Limitaciones

1. Las pruebas de hombre en el medio se hicieron contra el borde local con certificado autofirmado; una autoridad pública reforzaría la validación automática, pero no cambia la conclusión.
2. El objetivo de inyección de segundo orden no alcanzó su veredicto en esta corrida por el tope de tiempo; se apoya en la corrida previa (`F6-04`) y en las cargas manuales.
3. No se probó un ataque de hombre en el medio **interno** real (entre contenedores); se documenta la ausencia de TLS interno como debilidad conocida (B-30, B-32).
4. No se ejecutaron pruebas activas contra Supabase Auth ni contra ningún otro servicio de terceros.

8. Conclusiones

El sistema resiste los dos ataques pedidos. La **integridad** de los datos frente a la inyección SQL se sostiene en el ORM parametrizado y la validación en el servidor, con el cortafuegos como capa adicional: ningún parámetro resultó inyectable y ninguna carga alteró la base. La **confidencialidad** y la **autenticidad** frente al hombre en el medio se sostienen en TLS 1.2/1.3 con certificado validado, HSTS y los atributos de la cookie: un interceptor es detectado por cualquier cliente que valide, no se puede degradar ni despojar el cifrado, y las credenciales no viajan en claro por el tramo externo. La única debilidad es de cifrado interno entre contenedores (B-30, B-32), aceptada y mitigada por el aislamiento de red, con el cifrado interno mutuo como trabajo futuro.

9. Reproducibilidad

Cada medición conserva su comando exacto, su salida con los secretos enmascarados, la versión de la herramienta, el *commit* y la fecha, en `auditoria/evidencia/vulnerabilidades/` (`DS5-SQLI-NIVEL5`, `DS5-SQLI-MANUAL`, `DS5-MITM-INTERCEPTACION`) y se empaqueta en la entrega de auditoría. Las corridas se repiten con:

```
bash auditoria/scripts/sqlmap-nivel5.sh          # inyección SQL, nivel 5, ocho objetivos
bash auditoria/scripts/sqli-manual.sh            # cargas manuales, con y sin cortafuegos
bash auditoria/scripts/mitm-interceptacion.sh    # hombre en el medio: cinco vías
```